

模块六: Web 应用架构

第二十四讲: HOTL 实战——指挥 AI 完成结构化预测任务

[解释]

本讲概览

这是模块六的收官之战（实操 100%）。你将基于 L23 已调通的管道，参加多轮 AI 竞技赛。每轮比赛老师会推送一道未知题目，你需要在限定时间内调整 System Prompt，让大模型输出完美答案并回传大屏幕抢占榜单。这是对全部六周学习的综合检验——HTTP 调用、JSON 处理、API 对接、调用大模型、Prompt 工程——所有技能都将在此融会贯通。

一、竞赛规则

用自然语言指挥 AI，完成完整数据预测流程

任务背景：泰坦尼克号幸存预测

任务背景与数据

- **数据来源**：泰坦尼克号乘客记录（Kaggle 经典入门数据集）。训练集含幸存标签，测试集标签缺失，待你预测。
- **特征 (Features)**：Pclass, Sex, Age, SibSp, Parch, Fare, Embarked
- **目标 (Target)**：Survived（1 代表幸存，0 代表遇难）

极简工作流

1. **取卷**：无需你写网络代码，脚手架会提供 train.csv 和 test.csv 的下载地址。
2. **让 AI 代工**：使用 OpenCode / OpenCode 聊天框，向 AI 交代需求，让它为你生成一段加载并预测数据的 Python 机器学习代码。
3. **交卷**：用我们提供的 submission_client.py 脚本，将大模型生成的 CSV 自动抛向服务器！

操作规则

1. **无限次提交**：本节课操作期间，你可以无限次修改代码重新提交，系统只取你的最高 Accuracy（准确率）。
2. **角色约束**：本次实战的核心要求是：不得手写算法实现逻辑，只能通过自然语言向 AI 编程助手发出指令，让模型生成代码。
3. **迭代策略**：初版代码跑通后，可进一步指挥模型替换算法（如逻辑回归 → 随机森林）或添加特征工程，以提升准确率。

评分机制：服务端接收你的 submission.csv，自动计算并更新 Accuracy，大屏实时刷新排行榜。

[解释]

数据科学的基础范式验证

Titanic 生存预测是数据科学领域经典的基准数据集。该数据集保留了真实的工程挑战：例如核心特征缺失（Age）与枚举字符串的数值化处理（Sex）。在传统的开发流中，此类特征工程往往需要开发者手动编写繁杂的清洗代码。

引入大语言模型后，只要通过精确的 Prompt 建立约束，模型将全自动完成数据清洗、编码与训练逻辑。即使调试过程中产生错误情况，开发者也仅需反馈运行日志供模型自行修复。这种将精力从底层的控制流转移到高价值的架构层决策上，正是构建现代 HOTL（人在回路上）工作流的基础。

二、正式竞技

拿到开箱即用的基建模板

脚手架代码：结构说明

整体结构（保存为 arena_kaggle.py）

```
import pandas as pd
import requests, os

SERVER = "https://syb.ncu.edu.cn/smart-class/api"
ARENA_KEY = os.environ["ARENA_KEY"]

# == 第一部分：由 AI 生成（你来写 Prompt） ==
# 任务：
# 1. 从 SERVER/kaggle/train.csv 读取训练数据
# 2. 从 SERVER/kaggle/test.csv 读取测试数据
# 3. 训练模型并对测试集生成预测
# 4. 保存为 submission.csv
# 格式：两列 — PassengerId 和 Survived
#

# == 第二部分：提交（已写好，不要修改） ==
print("提交结果到服务器...")
with open("submission.csv", "rb") as f:
    resp = requests.post(
        f"{SERVER}/kaggle/submit",
        headers={"X-API-Key": ARENA_KEY},
        files={"file": f}
    )
print(resp.json())
# 返回示例：{"accuracy": 78.36, "best_accuracy": 78.36}
#
```

提交流程

你的电脑	课堂服务器
① GET /kaggle/train.csv	← 下载训练集（含标签）
② GET /kaggle/test.csv	← 下载测试集（无标签）
[AI 生成代码，本地运行]	
③ POST /kaggle/submit	→ 上传 submission.csv ← 返回 Accuracy
④ GET /kaggle/leaderboard	← 大屏实时轮询排名

第一部分是你的任务，第二部分已写好，不要修改。目标：让第一部分生成格式正确的
submission.csv，第二部分自动完成上传与评分。

三、人机协同实战

报错不是终点，而是 AI 的导航点

初始 Prompt 的构造：约束驱动指令设计

低效 Prompt（信息不足）

✗ "帮我写个预测代码"

模型无法确定：语言、库、数据路径、特征名称、输出格式——任何一项缺失都会导致生成的代码无法直接运行。

有效 Prompt 的四个约束维度

1. 数据来源：URL 或路径
2. 特征与目标：列名精确到字段
3. 算法与工具：指定库和模型类型
4. 输出规格：文件名、列名、格式

将此 Prompt 发给 OpenCode 或通义灵码，把生成的代码填入脚手架第一部分，运行后即完成数据下载→模型训练→结果提交的全流程。

参考初始 Prompt（可直接复制使用）

"请帮我写一段 Python 机器学习代码，填入已有脚本的空白处。

1. 用 pandas 从 `https://syb.ncu.edu.cn/smart-class/api/kaggle/train.csv` 读取训练数据，从 `https://syb.ncu.edu.cn/smart-class/api/kaggle/test.csv` 读取测试数据。
2. 特征列使用 `Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare`，目标列是 `Survived`。请自动处理缺失值（如 `Age` 填充中位数）和类别型特征（`Sex` 编码为 0/1）。
3. 用 sklearn 的逻辑回归模型进行训练。
4. 对 `test.csv` 进行预测，只导出 `PassengerId` 和 `Survived` 两列，保存为含表头的 `submission.csv`。"

[解释]

什么是“约束驱动的指令设计”？

大语言模型就像是一个能力极强但缺少业务上下文的实习生。如果不给予明确约束，它会凭直觉使用最常见的代码范式，但这往往与你当前的业务脚手架不兼容。

一个能够达成“One-shot”（一次通过）的工业级 Prompt，必须像产品需求文档一样包含四大明确纪律：

1. 获取的数据从哪里来（精确的接口 URL）
2. 需要用哪些维度的数据去干活（指定具体的特征列）
3. 采用什么样的技术栈（指定 sklearn 库）
4. 最终输出的成品长什么样（指定保存为包含特定两列的 CSV）

学会在指令中布下这些“规则结界”，防止 AI 自由发挥和产生幻觉，是人机协同开发模式中必须掌握的架构规划能力。

HOTL 迭代策略：两个典型场景

场景 A：代码报错

执行中 Python 抛出异常（如

`ValueError: Input contains NaN`）：

1. 将完整报错信息原文复制，作为后续 Prompt 的输入：

"运行上面的代码报错了，错误信息如下：[粘贴完整报错]。请分析原因并修复代码。"

2. 模型将定位错误并输出修复后的完整代码

要点：报错信息比自然语言描述更精确，是驱动模型修复的最有效输入。

场景 B：准确率偏低，需要提升

初版逻辑回归输出 78%，希望进一步优化：

- 向模型发出升级指令：

"当前逻辑回归准确率偏低。请保持数据读取与提交逻辑不变，仅将分类模型替换为随机森林

(`RandomForestClassifier`)，或添加基础特征工程，输出完整代码。"

HOTL 工作模式的本质：定义约束条件（数据来源、特征、输出格式）与提供反馈（报错日志、准确率结果）是你的职责；具体代码实现，是模型的职责。

[解释]

什么是 HOTL (Human-On-The-Loop) 模式

在传统的 HITL (Human-In-The-Loop, 人在回路中) 开发里，人类是编码链路上的执行者（亲自查错、手写调整参数）。但在现代的 HOTL (人在回路上) 范式中，人类退居为主管与裁判，AI 变成了底层代码劳动力。

遇到报错或需要升级算法精度时，千万不要习惯性地一行行 Debug。你的第一反应应该是：

1. 丢给系统抛出的错误日志，让 AI 去读并修复。
2. 给出上层策略变更（如“把原有的逻辑回归升级为考虑非线性的随机森林”），让 AI 直接吐出全套替换代码。这就是“控制方向和评估交给人类，代码执行交给模型”的核心协作观。

三、复盘：总结 HOTL workflow

从本次热身中提炼方法论

[解释]

技术复盘

竞赛结束后的复盘是学习闭环的关键环节。从无意识的测试到有意识的反思，你需要把实操中的感性体验转化为关于系统角色提示词（System Prompt）和黑盒测试的方法论。

复盘：你发现了哪些有效的人机协同策略？

架构师的必修课

1. 划定绝对边界 (Context Engineering)

- 不要让 AI 猜你在想什么！把下载 URL、特征列名、目标变量、甚至是你要用到的库（如 pandas, sklearn）一次性塞进初始 Prompt。

2. 将报错视为对话 (Feedback Loop)

- 传统程序员畏惧红色的语法 Error 提示。
- **HOTL 模式**：把 Error 一字不落扔给 AI。对于大语言模型来说，报错信息比你的文字描述更有价值，它是修正代码的最佳导航点。

3. 算法大拼盘 (Model Selection)

- 准确率卡在 80%？直接命令大模型："帮我把逻辑回归跑的代码，核心替换成'随机森林'，再给我一份完整版"。
- 你不用懂树是怎么分裂的，只要知道它的名字。

本质逻辑：提供精准的上下文

周次	我们的动作	核心目标
W3	写 AGENTS.md	让 OpenCode 知道"我有什么习惯"
W6-L22	写 PROJECT.md	让 OpenCode 知道"项目目录在哪"
W6-L24	面向 AI 写指令	指定数据来源、特征列名、缺失值处理策略与输出格式

💡 **统一视角**：这门课没有教你背诵机器学习语法，但你刚才完成了一个涵盖数据读取、模型训练、结果提交全流程的典型数据科学课后练习。这就是 **HOTL（人机协同）** 工作方式的实际体现。

[解释]

回顾：你和 AI 各自做了什么？

回想一下刚才的实操过程。你做的事情是：阅读报错信息、判断是缺失值没填还是列名对不上、决定要不要换一个更强的模型。而 AI 做的事情是：写出 pandas 的清洗代码、拼装 sklearn 的训练流水线、格式化输出 CSV。换句话说，你在做决策，AI 在做执行。这和传统的"遇到不会的语法就去搜索引擎查"完全不同——你从头到尾没有查过一行 API 文档，而是通过描述需求和反馈错误来驱动整个开发流程。

从第 3 周的 AGENTS.md 到上一讲的 PROJECT.md，再到今天用自然语言指挥 AI 写出完整的机器学习管道，这条线索始终一致：你的核心能力不是记住语法，而是能把业务需求准确地传达给机器。

模块六收官：你构建的能力图谱

四讲能力里程碑

讲次	能力里程碑
L21	理解 C/S 架构，完成第一次 API 调用
L22	将 Python 脚本封装为 Web 服务（FastAPI）
L23	串联课堂服务器与 LLM，构建完整数据处理管道
L24	以"指挥官"身份，驱动 AI 完成结构化数据处理任务

HOTL 三步工作法

1. **前期约束**：在首次 Prompt 中锁定数据来源、列名、算法与输出格式。
2. **中期迭代**：将报错日志原文反馈给模型，驱动模型自主修正。
3. **后期优化**：以自然语言指令驱动模型进行算法替换或特征工程。

展望：第 7、8 周

- **第 7 周**：AI 智能体开发——代码不仅处理数据，还能感知环境、规划行动。
- **第 8 周**：黑客松综合实战——今天的数据是刻意简化的热身；第 8 周面对的是真实业务数据的噪音与复杂度。

本次实战只是热身。第 8 周黑客松中，你将在更复杂、更贴近真实工业环境的任务里，再次验证并升级今天建立的工作流。

[解释]

本次实验的局限性

Titanic 数据集在互联网上被大量讨论和使用，大语言模型的训练语料中包含了许多针对该数据集的现成解法。因此当你用 Prompt 描述任务时，模型能快速给出可运行的代码，这并不意味着所有数据科学任务都能如此顺利。在第 8 周的黑客松中，数据集的字段数量、缺失模式和业务含义都会复杂得多，模型生成的代码大概率无法一次跑通。届时你需要反复执行"运行 → 阅读报错 → 反馈给模型 → 重新运行"这个循环。今天的练习价值在于：你已经完整地走过了这个循环，熟悉了它的节奏。后续面对更困难的数据时，流程是一样的，只是迭代次数会更多。